Java Inheritance — 15 Interview-Focused Codes

Beginner → Intermediate | Code + Output + Interview Points

Core idea: Inheritance allows a child class to acquire accessible members from a parent class. Java class inheritance uses **extends**.

1. Basic Single Inheritance

Basic inheritance: Developer inherits work() from Employee.

```
class Employee {
    void work() {
        System.out.println("Employee is working");
    }
}

class Developer extends Employee {
    void writeCode() {
        System.out.println("Developer writes code");
    }
}

public class Main {
    public static void main(String[] args) {
        Developer d = new Developer();
        d.work();
        d.writeCode();
    }
}
```

```
Output:
Employee is working
Developer writes code
```

Interview: Java class inheritance uses extends.

2. Inherited Variable

The child accesses an inherited non-private field.

```
class Person {
    String name = "Rahul";
}

class Student extends Person {
    void display() {
        System.out.println("Name: " + name);
    }
}

public class Main {
    public static void main(String[] args) {
        Student s = new Student();
        s.display();
    }
}
```

```
Output:
Name: Rahul
```

Interview: private parent members are not directly accessible in the child.

3. Parent and Child Methods

The child uses both inherited and its own methods.

```
class Employee {
    void work() {
        System.out.println("Employee works");
    }
}

class Manager extends Employee {
    void manage() {
        System.out.println("Manager manages the team");
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Manager m = new Manager();
        m.work();
        m.manage();
    }
}
```

```
 Output:
 Employee works
 Manager manages the team
```

Interview: inheritance supports code reuse.

4. Constructor Chaining with super()

super() calls the parent constructor.

```
class Employee {
    Employee() {
        System.out.println("Employee constructor");
    }
}

class Developer extends Employee {
    Developer() {
        super();
        System.out.println("Developer constructor");
    }
}

public class Main {
    public static void main(String[] args) {
        Developer d = new Developer();
    }
}
```

```
 Output:
 Employee constructor
 Developer constructor
```

Interview: parent constructor executes before child constructor.

5. Parameterized Parent Constructor

The child passes data to the parent constructor using super(name).

```
class Employee {
    String name;

    Employee(String name) {
        this.name = name;
    }
}

class Developer extends Employee {
    Developer(String name) {
        super(name);
    }

    void display() {
        System.out.println("Developer: " + name);
    }
}

public class Main {
    public static void main(String[] args) {
        Developer d = new Developer("Amit");
        d.display();
    }
}
```

```
 Output:
 Developer: Amit
```

Interview: if the parent has no no-argument constructor, the child must call a suitable parent constructor.

6. Method Overriding

The child provides its own implementation of an inherited method.

```
class Employee {
    void work() {
        System.out.println("Employee works");
    }
}

class Developer extends Employee {
    @Override
    void work() {
        System.out.println("Developer writes code");
    }
}

public class Main {
    public static void main(String[] args) {
        Developer d = new Developer();
        d.work();
    }
}
```

```
 Output:
 Developer writes code
```

Interview: overriding uses the same method signature in the child.

7. super Keyword — Parent Method

super.work() explicitly calls the parent version.

```
class Employee {
    void work() {
        System.out.println("Employee works");
    }
}

class Developer extends Employee {
    @Override
    void work() {
        super.work();
        System.out.println("Developer writes code");
    }
}

public class Main {
    public static void main(String[] args) {
        Developer d = new Developer();
        d.work();
    }
}
```

```
 Output:
 Employee works
 Developer writes code
```

Interview: super can access parent methods, fields, and constructors.

8. super Keyword — Parent Variable

super.role accesses the parent field when both classes have the same field name.

```
class Employee {
    String role = "Employee";
}

class Developer extends Employee {
    String role = "Developer";

    void display() {
        System.out.println("Child role: " + role);
        System.out.println("Parent role: " + super.role);
    }
}

public class Main {
    public static void main(String[] args) {
        Developer d = new Developer();
        d.display();
```

```
    }
}
```

```
Output:
Child role: Developer
Parent role: Employee
```

Interview: super is useful when parent and child members have the same name.

9. Multilevel Inheritance

Developer inherits Employee, and Employee inherits Person.

```
class Person {
    void eat() {
        System.out.println("Person eats");
    }
}

class Employee extends Person {
    void work() {
        System.out.println("Employee works");
    }
}

class Developer extends Employee {
    void code() {
        System.out.println("Developer writes code");
    }
}

public class Main {
    public static void main(String[] args) {
        Developer d = new Developer();
        d.eat();
        d.work();
        d.code();
    }
}
```

```
Output:
Person eats
Employee works
Developer writes code
```

Interview: Person → Employee → Developer is multilevel inheritance.

10. Hierarchical Inheritance

Multiple child classes inherit from one parent.

```
class Employee {
    void login() {
        System.out.println("Employee logged in");
    }
}

class Developer extends Employee {
    void code() {
        System.out.println("Developer codes");
    }
}

class Tester extends Employee {
    void test() {
        System.out.println("Tester tests software");
    }
}

public class Main {
    public static void main(String[] args) {
        Developer d = new Developer();
        Tester t = new Tester();

        d.login();
        d.code();
        t.login();
        t.test();
    }
}
```

```
Output:
```

```
Employee logged in
Developer codes
Employee logged in
Tester tests software
```

Interview: this is hierarchical inheritance.

11. Runtime Polymorphism

A parent reference points to different child objects.

```
class Employee {
    void work() {
        System.out.println("Employee works");
    }
}

class Developer extends Employee {
    @Override
    void work() {
        System.out.println("Developer writes code");
    }
}

class Tester extends Employee {
    @Override
    void work() {
        System.out.println("Tester tests software");
    }
}

public class Main {
    public static void main(String[] args) {
        Employee e;

        e = new Developer();
        e.work();

        e = new Tester();
        e.work();
    }
}
```

```
Output:
Developer writes code
Tester tests software
```

Interview: overridden method selection at runtime is dynamic method dispatch.

12. Upcasting

A child object is assigned to a parent reference.

```
class Employee {
    void work() {
        System.out.println("Employee works");
    }
}

class Developer extends Employee {
    void code() {
        System.out.println("Developer codes");
    }
}

public class Main {
    public static void main(String[] args) {
        Employee e = new Developer();
        e.work();
    }
}
```

```
Output:
Employee works
```

Interview: upcasting is automatic; the reference can directly access only members of the parent type.

13. Downcasting

A parent reference is cast back to the actual child type.

```
class Employee {
    void work() {
        System.out.println("Employee works");
    }
}

class Developer extends Employee {
```

```
    void code() {
        System.out.println("Developer codes");
    }
}

public class Main {
    public static void main(String[] args) {
        Employee e = new Developer();

        Developer d = (Developer) e;

        d.work();
        d.code();
    }
}
```

```
Output:
Employee works
Developer codes
```

Interview: incorrect downcasting can cause ClassCastException.

14. instanceof with Inheritance

instanceof checks type compatibility before casting.

```
class Employee {
}

class Developer extends Employee {
}

public class Main {
    public static void main(String[] args) {
        Employee e = new Developer();

        if (e instanceof Developer) {
            Developer d = (Developer) e;
            System.out.println("Object is a Developer");
        }
    }
}
```

```
Output:
Object is a Developer
```

Interview: instanceof is commonly used to make downcasting safer.

15. Combined Interview Example

Combines inheritance, constructors, super(), overriding, parent references, and runtime polymorphism.

```
class Employee {
    String name;

    Employee(String name) {
        this.name = name;
    }

    void work() {
        System.out.println(name + " is working");
    }
}

class Developer extends Employee {
    Developer(String name) {
        super(name);
    }

    @Override
    void work() {
        super.work();
        System.out.println(name + " is writing Java code");
    }
}

class Tester extends Employee {
    Tester(String name) {
        super(name);
    }

    @Override
```

```
    void work() {
        System.out.println(name + " is testing software");
    }
}

public class Main {
    public static void main(String[] args) {
        Employee e1 = new Developer("Rohan");
        Employee e2 = new Tester("Neha");

        e1.work();
        e2.work();
    }
}
```

```
Output:
Rohan is working
Rohan is writing Java code
Neha is testing software
```

Interview: this combines several of the most common inheritance questions in one program.

Inheritance Quick Revision

extends → class inheritance
super() → parent constructor
super.method() → parent method
super.variable → parent field
Overriding → child changes inherited method behavior
Upcasting → Parent ref = new Child()
Downcasting → Child ref = (Child) parentRef
instanceof → checks type compatibility
Runtime polymorphism → overridden method chosen at runtime

Interview question: Can Java support multiple inheritance with classes?

No. A Java class cannot extend two classes at the same time. Java uses interfaces for multiple-inheritance-like behavior.